

Report

Initial commit

我把 network.ortools.out 與 network.greedy.out 使用新的 network.graph 重新時做出來。

Second commit

完成整個 lab4 的 code 跟寫好 task1,2 的 report

Task1

先讀 network.graph 建立連線，分別先讀 GS 數衛星數與連接數，把每條連接用對應的地面站與衛星的 ID 存 datarate 到 linkDataRate 這個 mapping 中。讀 network.ortools.out 或 network.greedy.out，先把第一行的時間讀出來，gsToSat 存的是地面站要傳輸的衛星 ID，大小設為地面站的數量，把.out 的 2 到 21 行共 20 個地面站與衛星的連線讀出存進 gsToSat。

```
std::map<std::pair<int,int>, int> linkDataRate; //存GS_ID、SAT_ID對應的data rate
std::vector<int> gsToSat; //存GS_ID會傳的SAT_ID 大小是GS數量
```

```
std::ifstream fin("network.graph");
int numGs, numSat, numLinks;
fin >> numGs >> numSat >> numLinks;
for (int i = 0; i < numLinks; ++i){
    int gsId, satId, rate;
    fin >> gsId >> satId >> rate;
    linkDataRate[{gsId, satId}] = rate;
}
fin.close();

std::ifstream fin2(inputFile);
double totalTime;
fin2 >> totalTime; //.out裡面第一行是時間 先讀出來
gsToSat.resize(numGs);
for (int i = 0; i < numGs; ++i) {
    int gsId, satId;
    fin2 >> gsId >> satId;
    gsToSat[gsId] = satId;
}
fin2.close();
```

Task2.1

先拿出現在要傳輸的地面站 id 與衛星 id 對應的 datarate，並且

utNet = utCh.Install (satellites, groundStations) 所以衛星的 index 是 id 不用更改，而地面站的 index 會是衛星數加地面站 id，因 install 是先衛星在地面站。

雙向更改 datarate，把 gsStarted 設為 true，存的是地面站有沒有開始傳輸，並把地面站開始傳輸的時間記錄起來存到 gsStartTime，把現在要傳到的衛星 id 對應的 ip 拿出來。建立一個 socket 物件裡面存傳輸的接收端(衛星的 ip, port)，設定 MaxBytes = 125000, SendSize = 512，並把建立好的 socket 安裝到要開始傳輸的地面站上進行傳輸。用 bulkApp 取出 srcApp 傳輸的應用，並在真正開始傳輸封包在紀錄時間拿出真正傳輸花的時間，用 HandlePacketTx 去紀錄時間並把地面站改成開始傳輸。

```
std::vector<bool> gsStarted; //用來存GS有沒有傳送封包了
std::vector<double> gsStartTime; //存GS的傳輸開始時間
std::vector<int> satCompletedCount; //跟衛星完成傳輸的地面站數量
Ipv4InterfaceContainer utInterfaces; //存IP
```

```
void SendPacket(int gsId, int satId){
    // Task 2.1: Complete this function
    int datarate = linkDataRate[{gsId, satId}]; //取出現在GS跟SAT連線的data rate
    std::ostringstream oss;
    oss << (uint64_t)datarate << "kbps";
    uint32_t gsindex = satellites.GetN() + gsId; //GS的index
    utNet.Get(satId)->GetObject<MockNetDevice>()->SetDataRate(DataRate(oss.str()));
    utNet.Get(gsindex)->GetObject<MockNetDevice>()->SetDataRate(DataRate(oss.str()));
    Ipv4Address satAddr = utInterfaces.GetAddress(satId); //SAT的IP

    BulkSendHelper bulkSender("ns3::TcpSocketFactory", InetSocketAddress(satAddr, port));
    bulkSender.SetAttribute("MaxBytes", UintegerValue(125000)); //傳輸到達最大值會停止
    bulkSender.SetAttribute("SendSize", UintegerValue(512));
    ApplicationContainer srcApp = bulkSender.Install(groundStations.Get(gsId));
    srcApp.Start(Seconds(0.0)); //在GS_ID node上用tcp連到衛星ip，並開始傳輸
    //用bulkApp取出srcApp第一個應用(現在只有一個應用即傳輸)，當真正有呼叫tx把封包傳出去再紀錄時間
    Ptr<BulkSendApplication> bulkApp = DynamicCast<BulkSendApplication>(srcApp.Get(0));
    bulkApp->TraceConnectWithoutContext("Tx", MakeBoundCallback(&HandlePacketTx, gsId));
}
```

```
static void HandlePacketTx(int gsId, Ptr<const Packet> packet) {
    if (!gsStarted[gsId]) {
        gsStartTime[gsId] = Simulator::Now().GetSeconds();
        gsStarted[gsId] = true;
    }
}
```

Task2.2

`gsStarted` 存地面站有沒有開始傳輸，一開始預設為 `false` 還沒；`gsStartTime` 存地面站開始傳輸的時間，預設為 0；`gsEndTime` 存結束傳輸的時間，預設為 0；`satIdle` 存衛星有沒有在接收地面站的傳輸，開始預設衛星都是 `idle` 的，有地面站連接上並傳輸會改成非 `idle`；`satTotalTime` 存單一衛星總共的傳輸時間，預設為 0。從地面站 `id` 小到大，拿出該地面站要傳輸的衛星 `id`，如果地面站還沒開始傳輸跟衛星是空閒可接收的話就進行傳輸，並把衛星改成非空閒。

```
gsStarted.assign(numGs, false); //預設一開始GS都還沒發送封包
gsStartTime.assign(numGs, 0.0); //預設GS傳輸開始時間=0
gsEndTime.assign(numGs, 0.0); //預設GS傳輸結束時間=0
satIdle.assign(numSat, true); //預設衛星一開始都是空閒的
satTotalTime.assign(numSat, 0.0); //預設衛星總時間=0
satCompletedCount.assign(numSat, 0); //衛星完成接受的次數=0
```

```
// Task 2.2: Call SendPacket()
gsStarted.assign(numGs, false); //預設一開始GS都還沒發送封包
gsStartTime.assign(numGs, 0.0); //預設GS傳輸開始時間=0
gsEndTime.assign(numGs, 0.0); //預設GS傳輸結束時間=0
satIdle.assign(numSat, true); //預設衛星一開始都是空閒的
satTotalTime.assign(numSat, 0.0); //預設衛星總時間=0

for (int gsId = 0; gsId < numGs; ++gsId) {
    int satId = gsToSat[gsId];
    if (!gsStarted[gsId] && satIdle[satId]) { //如果GS還沒發送跟衛星是空閒的就發送封包
        SendPacket(gsId, satId);
        satIdle[satId] = false; //把衛星改成非idle
    }
}
```

Final commit

修改前面 lab4 有出錯的 code，我把傳輸時間改成看真正封包開始傳輸的時間，並且把接收的量改成要每次都倍數增加，跟把 report 剩下的 task3,4 與比較完成

Task3

利用 `GetNodeId` 拿出現在 `node` 的 `id`，如果是衛星的話把拿出來的 `id` 存到 `satId` 中，其餘不用考慮，因為 `GetTotalRx` 是累積衛星收到的總量，所以如果是第二次地面站完成傳輸，衛星應該要是接收到 125000×2 ，我用 125000 乘以完成次數加一當要看的總量 `threshold`。用 `GetTotalRx` 來看這次傳輸衛星收到的 `byte` 有沒有達到 `threshold`，有的話代表地面站把 125000byte 傳輸給衛星且衛星也完整收到，有的話就去找是是哪一個衛星已經完成傳輸，用現有的衛星 `id` 去找可以傳到這個衛星的地面站 `id` 有哪些，檢查是不是已經開始傳輸與結束時間還沒被記錄過(因還沒傳輸的話不可能是已經完成的，已經記錄過結束時間代表更早就完成的)，都符合就是要找的地面站 `id`，存到 `finishedGS` 中。

```
static void EchoRx(std::string context, const Ptr< const Packet > packet, const
// Task 3: Complete this function
int nodeId = std::stoi(GetNodeId(context)); //可以拿到現在node的index
int numSat = satellites.GetN();
if(nodeId >= numSat) return; //如果index大於衛星數量，代表是GS的index不用考慮
int satId = nodeId;
int completed = satCompletedCount[satId];

uint64_t threshold = 125000 * (completed + 1);
Ptr<PacketSink> sinkApp = DynamicCast<PacketSink>(sinkApps.Get(satId));
if (sinkApp->GetTotalRx() < threshold) {
    return;
}

int finishedGs = -1; //存結束傳輸的gs index
for(int gsId = 0; gsId < (int)gsToSat.size(); ++gsId){
    if(gsToSat[gsId] == satId && gsStarted[gsId] && gsEndTime[gsId] == 0.0){
        finishedGs = gsId; //找GS傳輸的SAT ID有沒有一樣、有沒有開始傳輸跟結束時間=0
        break;
    }
}
if(finishedGs < 0){
    return;
}
```

把結束的時間存到 `gsEndTime` 中，把結束時間減去開始時間為此地面站傳輸的總時間，加到 `satTotalTime` 中，因結束傳輸所以把該衛星設回 `idle`，把衛星完成傳輸的次數加一。看地面站 `id` 從小到大找還沒開始傳輸的且該地面站的目標衛星是現在完成的衛星，確認衛星是 `idle` 的，就接著進行傳輸並把衛星設回非 `idle`。

```

gsEndTime[finishedGs] = Simulator::Now().GetSeconds(); //紀錄結束傳輸時間
satTotalTime[satId] += gsEndTime[finishedGs] - gsStartTime[finishedGs];
satIdle[satId] = true;
satCompletedCount[satId]++;
for(int gsId = 0; gsId < (int)gsToSat.size(); ++gsId){
    if(!gsStarted[gsId] && gsToSat[gsId] == satId && satIdle[satId]){
        SendPacket(gsId, satId); //如果gs還沒開始傳輸跟衛星是idle的就進行傳輸
        satIdle[satId] = false;
        break;
    }
}
}

```

Task4

用 totalCollectionTime 存全部衛星傳輸最久的時間，把個別衛星的 satTotalTime 拿出來找最大的，並輸出在第一行。之後把每個衛星個別所花的時間依序印出來，依序把所有地面站開始傳輸的時間與結束的時間印出來。

```

// Task 4: Output File
std::ofstream fout(outputFile);
double totalCollectionTime = 0.0;
for(double t : satTotalTime){ //找全部中最大的傳輸時間
    if(t > totalCollectionTime){
        totalCollectionTime = t;
    }
}
fout << totalCollectionTime << "\n";

for(int satId = 0; satId < (int)satTotalTime.size(); ++satId){
    fout << satId << " " << satTotalTime[satId] << "\n";
}

for(int gsId = 0; gsId < (int)gsToSat.size(); ++gsId){
    fout << gsId << " " << gsStartTime[gsId] << " " << gsEndTime[gsId] << "\n";
}
fout.close();

```

Result question

1.

我 greedy 的時間是 0.387528，我 ortools 的時間是 0.153884 可以發現雖然傳輸時間都比 lab3 的模擬還長，但 greedy 的總傳輸時間仍比 ortools 的長。

2.

用 Lab3 計算出來的 greedy 時間是 0.193835 ns3 的是 0.387528

Ortools 是 0.0670526 ns3 是 0.153884 可以發現 lab3 方法算出來的時間比較短

因用 lab3 計算的 network.out 是只計算要傳輸的總 data 除掉 datarate 當作時間，沒有考慮額外 delay 與建立連線等花費的時間，但 ns3 模擬是實際的封包傳輸，會有 header 讓傳輸的總量變大，跟 tcp 傳輸會要等帶對端回傳 ack 才會開始傳輸，與 tcp 的 slow start 會讓傳輸的速度慢慢增加不會一開始就是以 datarate 傳輸，因此整體 lab4 中用 ns3 模擬出來的時間會比 lab3 的還長。

3.

MaxBytes 代表地面站向衛星傳輸的最大的 bytes 數量，當地面站傳送完該數量之後會停止傳輸；SendSize 是地面站每次交給 tcp 傳輸的 bytes 數量，類似把全部要傳的分割成一段一段傳給 tcp 傳送，tcp 會接收這些資料並轉換成 packet 進行傳送。此 code 地面站會不斷傳 512bytes 給 tcp 直到達到 MaxBytes

4.

我把 MaxBytes 增加到 250000，同時也把 GetTotalRx()要接收到 250000 才會停止，ortools 的時間變為 0.231146，greedy 的時間變為 0.610876，對比於原本 125000 都有略為增加。把 MaxBytes 增加到 500000，GetTotalRx()要接收到 500000，ortools 的時間變為 0.385669，greedy 的時間變為 1.05751。可以發現當要傳輸的量變大，傳輸時間也會跟著增多。